

# Single-Size Training, Benchmark-Scale Generalization: Multi-Head Dynamic Neural Guidance for Ant Colony Optimization

Dat Thanh Tran

College of Engineering and Computer Science, VinUniversity

Van Khu Vu

College of Engineering and Computer Science, VinUniversity

Yining Ma

Laboratory for Information and Decision Systems, MIT

## Abstract

Neural routing solvers are commonly evaluated on synthetic instances drawn from the same generator and scale regime used during training. This convention measures scalability, but it can overstate practical generalization: a solver may perform well on uniform in-distribution tests while failing to transfer to heterogeneous curated instances. We revisit learning-guided ant-colony search from this generalization-centered perspective. The proposed framework trains on a single synthetic size and is evaluated zero-shot on broad routing libraries. Unlike static neural guidance that emits one heatmap before search begins, it observes the evolving pheromone field and reference solution, then injects state-conditioned guidance throughout the search trajectory. We further introduce a multi-head variant in which a shared state encoder produces several head-specific heatmaps used by different ant groups. This gives the colony several learned search biases within one shared search process, improving robustness without requiring multiple encoders or separate ACO executions. Our evaluation follows a single-model transfer setting: train once at one synthetic size and test zero-shot on full benchmark libraries, reporting optimality gap, compute cost, and reliability. On full TSPLIB and CVRPLIB transfer tests, a model trained only at  $n = 1000$  improves over the matched single-head dynamic-guidance baseline on both problems. The result supports a generalization view of neural ACO: the learned component is most effective when it adapts a solver state during inference and when population diversity is preserved as distinct ant-level priors rather than compressed into one heatmap.

## 1 Introduction

Learning-guided optimization aims to combine the robustness of handcrafted solvers with the adaptability of neural models. In vehicle routing, this direction is attractive because classical heuristics such as LKH and HGS contain decades of algorithmic knowledge, while neural models can learn instance-dependent decision rules that are difficult to specify by hand. However, the dominant experimental convention in neural routing solvers has a structural weakness: models are trained and tested on synthetic instances drawn from narrow generators, often with separate models or tuned settings for each scale. Such protocols establish that a method scales on controlled distributions, but they do not establish whether one learned rule transfers to the diverse, clustered, geographic, and irregular instances that motivate practical routing solvers.

This paper makes transfer to curated instances the primary evaluation target. Following recent neural routing surveys [15], we distinguish *synthetic scalability* from *zero-shot in-problem generalization*. The former asks whether a method can run on large generated instances; the

latter asks whether a single trained model remains effective on heterogeneous curated libraries without fine-tuning. We therefore organize the study around a single-size training protocol: train on uniform synthetic instances at one size and evaluate on broad benchmark libraries without fine-tuning. The reported full-library evidence uses  $n = 1000$  training for both unconstrained and capacitated routing, while the  $n = 500$  runs are kept as a train-size ablation. The main reported metrics are optimality gap, compute cost, and reliability.

The method is well suited to this setting because it does not replace the underlying heuristic search. It intervenes inside the colony process as a state-conditioned guidance signal. Standard neural ant-colony methods train a network to emit a static edge-prior heatmap from the instance geometry, then keep that heatmap fixed while the pheromone process evolves. This creates a training-inference mismatch: the model is trained without observing the solver state but deployed inside a long-horizon stochastic search process. Our dynamic formulation treats neural guidance as a semi-Markov decision process. At periodic guidance steps, the policy observes the candidate graph, pheromone distribution, and incumbent solution, then emits edge-level guidance scores used by the ACO search process for the next group of sampling iterations. This dynamic interface allows the model to learn rules that depend on search progress, including when to reinforce promising regions and when to counteract pheromone stagnation.

This work adds a second mechanism: multi-head dynamic guidance. ACO is a population method, and its search quality depends on the diversity of ants. A single learned heatmap can improve the colony’s average bias, but it can also concentrate all ants around the same learned preference. The multi-head variant therefore gives the shared encoder  $K_h$  lightweight decoders, each producing a distinct heatmap. The  $m$  ants are partitioned across these heads, so one ACO run explores several learned priors in parallel. The population update remains shared: all heads are sampled in the same colony update, and the neural overhead is dominated by the shared encoder rather than by repeated model calls.

Our contributions are:

1. We recast dynamic neural guidance as a generalization-centered learning-guided solver: one model is trained at a single synthetic size and tested zero-shot on full benchmark libraries.
2. We introduce a multi-head dynamic guidance mechanism that assigns different learned heatmaps to different ant groups while preserving shared encoding and a single population update.
3. We provide a transfer evaluation protocol reporting gap, runtime, and reliability, separating library-scale generalization from conventional synthetic-scale testing; the reported full-library results cover 228 TSPLIB-derived and 110 CVRPLIB-derived instances.
4. We retain the original dynamic guidance mechanisms, namely state-aware representation, trajectory-aware training, and scope-restricted refinement, but reposition them as the reason the learned rule can transfer through a strong heuristic backbone.

## 2 Related Work

**NRSs as learned heuristics.** Recent surveys argue that neural routing solvers are heuristic algorithms in which learned rules replace handcrafted rules inside construction, improvement, decomposition, or population search frameworks [15]. This heuristic view is important for positioning the proposed intervention. The goal is not to compare a learned ACO intervention indiscriminately against every neural solver category, since different categories use different amounts of solver structure, domain knowledge, and test-time computation. The most direct question is whether a learned rule improves the heuristic framework into which it is inserted. For this reason, the central internal comparison in this paper is between unguided ACO search,

single-head dynamic guidance, and multi-head dynamic guidance under the same inference budget.

**Conventional scalability evaluation and its limitation.** The conventional NRS evaluation pipeline commonly tests models on synthetic instances with fixed scales and distributions. This setting is useful for measuring scalability, but it can conflate three distinct regimes: in-distribution performance, cross-scale transfer, and zero-shot generalization to diverse benchmark instances. The survey’s generalization-focused protocol addresses this by evaluating single-model performance on broad benchmark libraries, using effectiveness, efficiency, and reliability as complementary metrics [15]. It also highlights that conclusions under this protocol can differ sharply from synthetic-scale results: several NRSs that look strong on uniform generated instances lose reliability or fall below simple construction heuristics on benchmark distributions. Our experimental framing adopts this distinction. Synthetic scaling remains a computational stress test, while benchmark-library transfer is the main evidence for generalization.

**Existing strategies for in-problem generalization.** The survey identifies two broad design responses to in-problem generalization. One line improves the model interface, for example by allocating more decoding or attention capacity to dynamic relationships among partial solutions, remaining nodes, or local neighborhoods. These methods can improve state awareness but often increase inference cost because the model repeatedly re-embeds changing decision states. Another line injects stronger geometric or distance-based bias into the neural policy, which helps on routing objectives but can tie the method to specific distance structures or handcrafted modules. A separate data-side strategy changes the test or training distribution: decomposing large instances into train-scale subproblems, reducing the search space, normalizing coordinates, or training on more diverse distributions. These approaches improve transfer in many settings, but they can also localize the problem too aggressively, lose global context, or move the burden to data selection.

**Improvement and population search.** Improvement-based and population-based NRSs are especially relevant because they retain an iterative solver and learn one of its decision rules. Large-neighborhood methods learn destroy, repair, or region-selection criteria; neural ACO methods learn edge preferences used by a pheromone-guided population process. The survey notes that within the same heuristic category, learned rules can still provide useful implicit knowledge complementary to handcrafted rules, even when current NRSs underperform strong heuristics in absolute terms [15]. Our method follows this category-aware interpretation. It does not claim that neural guidance replaces the solver. It studies whether a learned state-conditioned prior improves a perturbative population search process while preserving feasibility, local projection, and iterative improvement.

**Neural-guided ACO and the remaining gap.** ACO combines pheromone memory with heuristic visibility. DeepACO, GFACS, GTG-ACO, and HeatACO learn neural priors for the ACO transition rule, but the learned prior is typically static. Once emitted, the heatmap does not adapt to pheromone concentration, incumbent structure, or stagnation. This leaves a gap between the survey’s desired generalization properties and the dominant neural-ACO interface. A generalizable neural ACO method should observe the same solver state that drives ACO’s test-time behavior, should avoid repeated expensive re-embedding at every constructive decision, and should preserve the population diversity that makes ACO robust. The dynamic interface addresses the first two requirements through state-conditioned guidance and amortized trajectory-level updates. The multi-head extension addresses the third by mapping multiple learned priors to different ant groups inside one shared pheromone process.

### 3 Dynamic Neural ACO

We consider Euclidean TSP and capacitated CVRP instances represented by a sparse  $k$ -nearest-neighbor candidate graph. At ACO iteration  $t$ , an ant at node  $i$  samples a candidate successor  $j$  using pheromone  $\tau_{ij}$ , heuristic visibility  $\eta_{ij}$ , and neural guidance  $z_{ij}$ :

$$\log p_{\theta}(j \mid i, t) = \alpha \log \tau_{ij}^{(t)} + \beta \log \eta_{ij} + \gamma z_{ij}^{(h)}, \quad (1)$$

where  $h$  indexes the current neural guidance period. The neural guidance scores are held fixed for several search iterations and refreshed when the policy observes a new macro-state.

#### 3.1 Semi-MDP Formulation

The macro-state at guidance step  $h$  is

$$\mathcal{S}_h = (G, \tau^{(h)}, b^{(h)}), \quad (2)$$

where  $G$  is the static candidate graph,  $\tau^{(h)}$  is the pheromone field, and  $b^{(h)}$  is the incumbent solution. The policy action is a sparse edge-prior  $z_{\theta}(\mathcal{S}_h)$ . The environment then runs  $S$  ACO sampling iterations with scope-restricted refinement and pheromone updates, producing the next macro-state. Training minimizes the expected cumulative post-refinement cost across the trajectory rather than a one-shot constructive loss. This aligns training with the solver behavior used at inference.

#### 3.2 Scope-Restricted Refinement

The search process is perturbation based. Each ant starts from the incumbent solution and samples a small set of edge changes rather than constructing a full tour from scratch. The resulting local disruption is repaired by SRR, which applies local search only in the neighborhood affected by the perturbation. This preserves the causal relation between sampled edges and post-refinement reward while keeping cost proportional to the perturbation size and candidate degree, not to the full instance size. For CVRP, the same principle is used with capacity-aware route operations.

#### 3.3 State-Aware Guidance

The policy encodes local edge features derived from distance, pheromone level, pheromone concentration, incumbent adjacency, and neighborhood statistics. These features are processed by a graph neural encoder over the sparse candidate graph. A decoder emits one scalar logit per candidate edge. The logit is not a replacement for the ACO heuristic; it is an additive residual in log transition space. This keeps the classical distance and pheromone signals active while allowing the learned model to reshape search according to the observed state.

We instantiate the state representation with a 12-layer message-passing encoder on the sparse candidate graph. For TSP, node features are the two coordinates. For CVRP, node features are coordinates, normalized demand, and a depot indicator. Both problems use six edge features when all dynamic channels are enabled: normalized distance, pheromone dispersion, relative log-pheromone, incumbent successor indicator, incumbent predecessor indicator, and new-edge indicator. The same representation-to-prior interface is used for both problems; problem-specific feasibility is handled by the search process.

#### 3.4 Feature Design and Scale Invariance

The state representation is designed to transfer from one training size to benchmark instances of different scales. The policy never receives a raw dense distance matrix or an absolute instance-size embedding. Instead, every edge feature is defined locally on the  $k$ -nearest-neighbor graph

and normalized against the source node’s candidate neighborhood. This gives the policy a local view of search state whose statistics are less sensitive to whether the instance has hundreds, thousands, or tens of thousands of nodes.

For each directed candidate edge  $(i, j)$ , the feature vector contains three static or slowly varying geometric terms and three dynamic search terms. The normalized distance feature is

$$x_{ij}^{\text{dist}} = \frac{d_{ij}}{\frac{1}{k} \sum_{\ell \in \mathcal{N}_k(i)} d_{i\ell}}, \quad (3)$$

which measures whether  $j$  is unusually close or far relative to node  $i$ ’s local candidate set. The pheromone-level feature is

$$x_{ij}^{\text{rel}\tau} = \log \frac{\tau_{ij}}{\frac{1}{k} \sum_{\ell \in \mathcal{N}_k(i)} \tau_{i\ell}}, \quad (4)$$

clamped for numerical stability. The pheromone-dispersion feature is node-level:

$$x_i^{\text{cv}\tau} = \frac{\sqrt{\frac{1}{k} \sum_{\ell \in \mathcal{N}_k(i)} (\tau_{i\ell} - \bar{\tau}_i)^2}}{\bar{\tau}_i + \epsilon}. \quad (5)$$

It measures whether the ACO process has locally converged around a few edges or remains diffuse. Finally, two binary incumbent-adjacency features mark whether  $(i, j)$  is the current successor or predecessor edge in the incumbent solution, and a new-edge feature marks whether selecting  $(i, j)$  would perturb the incumbent topology.

This design is deliberately solver-state-centric. The model is not asked to infer a complete route from coordinates alone. It observes how a strong heuristic process is currently allocating pheromone and where the incumbent tour or route set is locally rigid. This distinction is central to the generalization claim: the learned component acts as an intervention on a normalized search state, not as a memorized global construction policy.

For CVRP, the same edge-feature template is applied to a route sequence in which the depot may appear multiple times. We therefore derive directed successor and predecessor relations from consecutive route entries rather than assuming a Hamiltonian cycle. Depot successors and predecessors are handled as sets, while customer successors and predecessors are stored as arrays. This preserves the meaning of incumbent-adjacency features under multiple routes.

### 3.5 Trajectory-Aware Optimization

Static neural-ACO training optimizes a one-shot objective:

$$\min_{\theta} \mathbb{E}_{\pi \sim p_{\theta}(\cdot | G)} [C(\pi)], \quad (6)$$

where the policy emits a heatmap once and ants sample tours without the pheromone history that will exist at inference. The dynamic objective replaces this with a trajectory objective over the actual search process. Let  $\hat{\pi}_{a,s}^{(h)}$  be the post-SRR solution produced by ant  $a$  during search iteration  $s$  under guidance step  $h$ . The training objective is

$$J(\theta) = \mathbb{E} \left[ \frac{1}{HSm} \sum_{h=1}^H \sum_{s=1}^S \sum_{a=1}^m C(\hat{\pi}_{a,s}^{(h)}) \right]. \quad (7)$$

Minimizing the area under the search curve encourages guidance that is useful throughout the run, not merely at the final iteration.

The log-probability of a perturbative ant trajectory decomposes only over stochastic edge choices. If ant  $a$  makes  $T_a$  stochastic decisions, its normalized log-probability is

$$\bar{\ell}_a = \frac{1}{T_a} \sum_{t=1}^{T_a} \log p_{\theta}(v_t | u_t, \mathcal{S}_h). \quad (8)$$

The per-decision normalization prevents trajectories with more stochastic choices from dominating PPO updates. For each search iteration, the baseline is the mean post-refinement cost across ants:

$$b_s = \frac{1}{m} \sum_{a=1}^m C(\hat{\pi}_{a,s}), \quad A_{a,s} = b_s - C(\hat{\pi}_{a,s}). \quad (9)$$

Lower-cost ants therefore receive positive advantage. The clipped PPO objective is

$$\mathcal{L}^{\text{PPO}}(\theta) = -\mathbb{E}_{a,s} \left[ \min \left( r_{a,s}(\theta) \hat{A}_{a,s}, \text{clip}(r_{a,s}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{a,s} \right) \right], \quad (10)$$

where

$$r_{a,s}(\theta) = \exp \left( \bar{\ell}_{a,s}^{\text{new}} - \bar{\ell}_{a,s}^{\text{old}} \right). \quad (11)$$

The replay buffer stores pheromone snapshots, incumbent successor arrays, compact action traces, feasibility masks, and post-refinement costs. It does not store the full graph neural computation graph. During PPO replay, the policy recomputes logits from the stored macro-state and evaluates the selected stochastic decisions under the current parameters. This is essential for scaling the training procedure to large sparse graphs.

Each stored trace contains the stochastic decisions made by each ant and the number of stochastic decisions used for normalization. The log-probability ratio is computed after dividing each ant’s trace log-probability by this decision count. This matters because perturbative ants may encounter different numbers of non-degenerate choices; without normalization, longer stochastic traces would dominate the PPO update. Advantages are computed from post-refinement costs by subtracting each ant’s cost from the mean cost of the relevant comparison group and are normalized unless disabled by ablation.

### 3.6 Perturbative Search and Credit Assignment

The search process starts each ant from the incumbent solution and applies a bounded number of relocation or reconnection steps. Each sampled move introduces at most a small number of new edges. The local-search phase then repairs only the region affected by those newly introduced edges. This is the role of SRR.

This design solves the credit-assignment problem that appears when neural policies are trained through powerful local search. If unconstrained local search rewrites the entire solution, the final cost may depend weakly on the neural choices that initialized the ant. If local search is truncated, the cost remains closer to the neural action but the resulting solution quality is artificially poor. SRR avoids this tradeoff by exploiting the incumbent structure. Before perturbation, the incumbent is locally optimized under the candidate graph. After perturbation, suboptimality is concentrated around the modified edges. Repairing that region to convergence is enough to restore local optimality under the candidate graph while preserving a causal link between the policy’s sampled edges and the final reward.

For TSP, the local operators are candidate-graph 2-opt moves triggered from the affected checklist. For CVRP, the same scope principle is used with capacity-aware route operations: intra-route 2-opt, inter-route relocate, inter-route swap, and 2-opt\* segment exchange. Capacity feasibility is handled by the search process, not by a problem-specific neural decoder. This separation is important for generalization. The neural model learns how to bias local perturbations, while the search process enforces feasibility and performs problem-specific repair.

### 3.7 Pheromone Stabilization

Classical max-min ant systems bound pheromone values as a function of incumbent cost. That coupling is useful for handcrafted solvers, but it creates non-stationary neural inputs: the same

edge can have different numerical meaning as solution cost changes. We therefore use fixed normalized bounds

$$\tau_{\max} = 1, \quad \tau_{\min} = \frac{1}{k}. \quad (12)$$

After each search iteration, the pheromone update is expressed as a convex interpolation:

$$\tau_{ij}^{(t+1)} \leftarrow (1 - \rho)\tau_{ij}^{(t)} + \rho\mathcal{T}_{ij}^{(t)}, \quad (13)$$

where  $\mathcal{T}_{ij}^{(t)} = \tau_{\max}$  if  $(i, j)$  belongs to the selected reinforcing solution and  $\tau_{\min}$  otherwise. This makes pheromone features comparable across scales and across training trajectories. This is not a cosmetic design choice; it is part of the generalization mechanism.

### 3.8 Inference-Time Guidance Schedule

Training uses constant guidance by default. Annealing is used only as a deployment-time control, because annealing the guidance weight during training reduces the policy-gradient signal in later search iterations. At inference, however, the guidance coefficient can be linearly annealed inside a guidance period:

$$\gamma_s = \gamma_{\max} \left(1 - \frac{s}{S-1}\right) + \gamma_{\min} \frac{s}{S-1}, \quad s = 0, \dots, S-1. \quad (14)$$

This schedule converts early neural steering into late pheromone exploitation. The same trained policy can also be deployed with warmup or phased injection: the search process may run for an initial set of outer steps without neural priors, after which the current solver state is encoded and dynamic guidance begins. These inference schedules are not separate trained policies. They are deployment-time controls applied to the same state-conditioned model.

## 4 Multi-Head Dynamic Guidance

Ant-colony search uses many ants because a single sampling bias is rarely sufficient. The same principle applies to learned priors. If all ants share one neural heatmap, the learned guidance can reduce diversity and make the colony overly dependent on one policy mode. The multi-head design introduces  $K_h$  head-specific priors while retaining the same dynamic state interface.

### 4.1 Architecture

The state encoder is shared:

$$H = f_{\theta}(\mathcal{S}_h). \quad (15)$$

Each head  $r \in \{1, \dots, K_h\}$  has a lightweight decoder

$$z^{(r)} = g_{\theta,r}(H, e_r), \quad (16)$$

where  $e_r$  is a learned head embedding. The result is a stack of heatmaps

$$Z_h = \{z^{(1)}, z^{(2)}, \dots, z^{(K_h)}\}. \quad (17)$$

Because the encoder dominates neural cost, multiple heads add only decoder-level overhead. We instantiate the head-conditioned decoder as a FiLM-style MLP. Each learned head code generates feature-wise scale and shift parameters for the decoder hidden layers, so the heads share decoder structure while producing distinct edge-prior logits. The decoder produces one sparse edge prior per head.

## 4.2 Ant-to-Head Assignment

The  $m$  ants are partitioned into  $K_h$  groups. Ants in group  $r$  use  $z^{(r)}$  in 1. The search process samples all groups in a single ACO batch and updates one shared pheromone field from the resulting colony. Thus, multi-head guidance is not an ensemble of separate solvers. It is a single colony whose ants receive different learned priors. The search process evaluates each head separately and dispatches ant groups to their assigned head; the heads are never averaged into one heatmap.

The ant allocation is a deployment variable, not a change in search budget. Uniform allocation is the natural balanced setting, while non-uniform allocation tests the tradeoff between preserving the strongest learned prior and reserving ants for alternative priors. The main full-library deployment uses 50, 20, 15, 15 under the same total count of  $m = 100$  ants. Half of the colony therefore follows non-primary heads, which makes the result a multi-head population test rather than a protected single-head test. The more conservative 97, 1, 1, 1 allocation is retained only as a diagnostic showing the upper end of protected-primary behavior. The equal 25, 25, 25, 25 allocation is reported where the transfer runs are available, because allocation sensitivity is part of the mechanism rather than a cosmetic hyperparameter.

## 4.3 Training Objective

The base objective remains the trajectory-level cost minimized by PPO. For multi-head training, ant costs are additionally grouped by head. Head competition weights favor heads whose ant group produces lower post-refinement costs, while an optional diversity term discourages all heads from collapsing to identical heatmaps. The head score can use either the mean cost of each head group or the mean of the best few ants in the group. The competition weight is a softmax over negative head scores:

$$w_r = \frac{\exp(-\gamma_h q_r)}{\sum_{r'} \exp(-\gamma_h q_{r'})}, \quad (18)$$

where  $q_r$  is the selected head cost statistic and  $\gamma_h$  controls how sharply training emphasizes the best-performing heads. The resulting objective has the form

$$\mathcal{L}_{\text{MH}} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}} + \lambda_{\text{head}} \mathcal{L}_{\text{head}}, \quad (19)$$

where  $\mathcal{L}_{\text{head}}$  is derived from relative head performance and  $\mathcal{L}_{\text{div}}$  is computed from head-output similarity. The default experiments use mixed-ant deployment: every search trajectory contains all heads. The diversity term is implemented as the average pairwise Jensen-Shannon divergence between the per-node softmax distributions induced by head priors. It is subtracted from the PPO loss with a small coefficient, so training rewards useful head differentiation without allowing diversity alone to dominate solution quality.

For the  $n = 1000$  transfer runs, the multi-head model is initialized from the matched single-head model and trained with a teacher anchor on the first head. The anchor keeps the first head close to the single-head dynamic prior while allowing the remaining heads to learn alternative perturbation biases. The objective becomes

$$\mathcal{L}_{\text{anchor}} = \mathcal{L}_{\text{MH}} + \lambda_{\text{anchor}} \left\| z_{\theta}^{(1)} - z^{\text{single}} \right\|_2^2, \quad (20)$$

where  $z^{\text{single}}$  is the prior emitted by the frozen matched single-head policy on the same search state. This anchor is not a separate inference model. It is a training constraint that protects the known useful dynamic prior while giving the colony additional learned modes.

## 5 Generalization-Centered Evaluation

### 5.1 Protocol

The main protocol trains one model on synthetic uniform instances of a single size and evaluates it without fine-tuning on benchmark libraries. The completed full-library protocol uses  $n = 1000$  for both TSP and CVRP. The  $n = 500$  setting is retained as a train-size ablation because earlier capacitated-routing results showed that it can be competitive on split benchmarks. The main benchmark sets are:

- unconstrained routing: full TSPLIB/survey benchmark set, using small, medium, and large buckets.
- capacitated routing: full CVRPLIB/survey benchmark set, using the same three size buckets.

Each method is evaluated with the same inference budget, candidate graph policy, and hardware measurement procedure. The reported metrics are average gap to the known best or reference solution, measured runtime, and reliability. Reliability records solved instances, timeouts, out-of-memory failures, and invalid outputs.

The experiments are organized around five claim-shaped questions. First, does dynamic guidance improve the same ant-colony process when the learned policy is evaluated outside its synthetic training generator? Second, does splitting the learned prior across ant groups improve the matched single-head version without increasing the ant budget? Third, which head allocation exposes the useful tradeoff between a reliable primary prior and auxiliary exploration? Fourth, how sensitive is transfer to the single training size? Fifth, when the method fails or slows down, is the bottleneck caused by the learned prior, by colony sampling, or by local projection? The tables below answer the first four questions with the available transfer runs; the mechanism section states the diagnostics needed for the fifth.

### 5.2 Methods Compared

The core comparisons are:

1. unguided ACO search.
2. Single-head DYNACO trained at  $n = 1000$ .
3. Multi-head MH-DYNACO trained at  $n = 1000$  with all heads active under the same total ant count.
4. Single-head and multi-head  $n = 500$  models as train-size ablations.
5. External neural and classical baselines from the survey benchmark, reported separately when inference settings differ.

### 5.3 Why the Main Protocol Changes from the Conference Version

The conference version primarily emphasized synthetic scale: train and test on generated routing distributions at large sizes, then report that dynamic guidance improves a strong ACO search process. That evidence remains useful, but it is no longer the center of the journal paper. Here, synthetic scaling becomes a secondary sanity check and benchmark transfer becomes the main experiment. This is a substantive change, not a cosmetic rearrangement.

The difference is summarized by three evaluation questions. First, synthetic scaling asks whether the solver can operate at large  $n$  under a controlled generator. Benchmark generalization asks whether the same trained model still helps on non-uniform, clustered, geographic,

and historically curated instances. Second, in-distribution synthetic testing can obscure the distinction between a learned heuristic and a learned generator-specific bias. Benchmark libraries reduce that risk because their structure is not produced by the same sampler used for training. Third, reporting only gap hides solver reliability. The protocol therefore records timeouts, invalid outputs, out-of-memory failures, and solved-instance counts.

This shift also changes the role of baselines. External NRS baselines from the survey are used to situate the result under a common generalization protocol. The most important internal baseline remains unguided ACO, because it isolates whether the learned guidance improves the exact heuristic framework into which it is inserted. The multi-head comparison is then evaluated under a matched solver, matched training size, and matched inference budget.

## 5.4 Dataset and Bucket Definitions

The experiments use the STAR/survey-converted benchmark files. The unconstrained-routing target is the full TSPLIB/survey collection with 228 instances, and the capacitated-routing target is the full CVRPLIB/survey collection with 110 instances. The protocol reports three size buckets for each variant: small instances below 1K nodes/customers, medium instances from 1K to below 10K, and large instances from 10K to 100K. When a source file uses a different split, such as the completed capacitated-routing sub-1K versus 1K-plus split, we keep the split explicit instead of manually approximating the three-bucket table.

The benchmark section distinguishes four data regimes:

1. synthetic training data, generated online from uniform coordinates;
2. synthetic scale tests, retained from the conference paper as controlled scalability evidence;
3. benchmark-library transfer tests, used as the main journal evidence;
4. per-instance appendix tables, used for auditability and failure analysis.

## 5.5 Training Configuration

The primary full-library training setting is one-size training at  $n = 1000$ . This setting is used for both problem families in the completed TSPLIB and CVRPLIB runs. The earlier  $n = 500$  results remain informative because they show that multi-head guidance can improve over a matched single-head model even when the training size is smaller than many benchmark instances. We therefore report  $n = 500$  as a train-size ablation rather than treating any single training size as universally optimal.

The shared solver and learning setting used by the completed full-library runs is:

- candidate degree  $k = 32$ ;
- ants  $m = 100$ ;
- training horizon  $H = 10$  with 100 inner search steps;
- evaluation horizon  $H = 10$  with 100 inner search steps;
- PPO learning rate  $5 \times 10^{-6}$ ;
- multi-head evaluation with four heads and ant allocation 50, 20, 15, 15.

All multi-head and single-head evaluations keep the same total number of ants. The multi-head allocation changes how the population is divided across learned priors, not the amount of search performed.

Table 1: Full benchmark-library transfer after training on one synthetic size,  $n = 1000$ . Gap is reported in percent; reliability is solved/total. All guided methods use the same total ant count and search horizon.

| Problem | Benchmark                | Single-head DyNACO $n = 1000$ |       |         | MH-DyNACO $n = 1000$ |        |         |
|---------|--------------------------|-------------------------------|-------|---------|----------------------|--------|---------|
|         |                          | Gap                           | Time  | Rel.    | Gap                  | Time   | Rel.    |
| TSP     | Full TSPLIB-derived set  | 2.060                         | 4.35s | 228/228 | <b>1.954</b>         | 10.79s | 228/228 |
| CVRP    | Full CVRPLIB-derived set | 2.975                         | 4.23s | 110/110 | <b>2.867</b>         | 4.98s  | 110/110 |

For synthetic capacitated-routing training, customer demands are sampled uniformly from  $\{1, \dots, 9\}$  and normalized by the capacity. The default capacity follows a scale schedule: 50 below 1K customers, 250 at 1K, 500 at 5K, 1000 at 10K, and 2000 at 50K and above. After normalization, capacity is one and route feasibility is enforced in normalized units. The neural network therefore sees normalized demand while route construction and local projection enforce capacity constraints.

## 5.6 Reliability and Failure Accounting

The survey framing emphasizes that a neural solver is evaluated not only by average gap on successful runs but also by reliability. The result table therefore includes a reliability column for every method and bucket. A run is counted as unsuccessful if it times out under the declared budget, runs out of memory, returns an infeasible solution, produces a gap above the survey failure threshold, or fails to produce an output. The reliability column keeps hard instances inside the reported protocol instead of averaging only successful outputs.

For the guided methods, reliability has two meanings. At the solver level, the search process is expected to return a feasible solution because feasibility is enforced by ACO construction and capacity-aware repair. At the neural-guidance level, reliability measures whether the learned prior improves or at least does not destabilize the search process across diverse benchmark distributions. We therefore report both solved/total and win/loss/tie counts against unguided ACO when per-instance outputs are available.

## 5.7 Primary Result Table

1 reports the completed full-library transfer cells under the matched  $n = 1000$  setting. On TSP, multi-head guidance with allocation 50, 20, 15, 15 reduces the average full-library gap from 2.060% to 1.954%. On CVRP, the same allocation reduces the average gap from 2.975% to 2.867%. The TSP gain comes with additional runtime, so the table should be read as a compute-quality tradeoff rather than a free improvement. The important comparison is budget matched: both variants use 100 ants, the same candidate degree, the same guidance horizon, and the same benchmark inputs.

## 5.8 Allocation Sensitivity

Multi-head guidance creates a new inference-time tradeoff: allocating too many ants to the primary head reduces the method toward a protected single-prior search, while allocating too few ants to it can dilute the strongest learned dynamic prior. 2 reports the completed allocation comparisons. The equal allocation is not dominated. On TSPLIB instances below 1K nodes, 25, 25, 25, 25 is slightly better and faster than 50, 20, 15, 15. On the full CVRPLIB-derived set, 50, 20, 15, 15 has the better mean per-instance gap, while 25, 25, 25, 25 has lower mean cost and lower measured runtime. This pattern supports the intended interpretation: the allocation controls a robustness-diversity tradeoff, and the best point depends on the problem family, split, and training history.

Table 2: Allocation sensitivity under the same total ant count. Lower gap, cost, and time are better. The full-TSPLIB 25, 25, 25, 25 aggregate is left open because the available run set does not contain a completed aggregate for that cell.

| Problem | Benchmark                  | Allocation     | Gap          | Mean Cost       | Mean Time    |
|---------|----------------------------|----------------|--------------|-----------------|--------------|
| TSP     | TSPLIB-derived, < 1K nodes | single-head    | 0.541        | 27544.29        | 0.69s        |
| TSP     | TSPLIB-derived, < 1K nodes | 25, 25, 25, 25 | <b>0.523</b> | <b>27518.79</b> | <b>0.67s</b> |
| TSP     | TSPLIB-derived, < 1K nodes | 50, 20, 15, 15 | 0.528        | 27519.86        | 2.52s        |
| CVRP    | Full CVRPLIB-derived set   | single-head    | 2.975        | 76.43           | 4.23s        |
| CVRP    | Full CVRPLIB-derived set   | 25, 25, 25, 25 | 2.891        | <b>76.37</b>    | <b>4.08s</b> |
| CVRP    | Full CVRPLIB-derived set   | 50, 20, 15, 15 | <b>2.867</b> | 76.40           | 4.98s        |

Table 3: Train-size and anchoring evidence on CVRPLIB transfer. The  $n = 500$  rows are split-weighted from sub-1K and 1K-plus runs. The  $n = 1000$  rows use the full 110-instance CVRPLIB-derived set.

| Model                                          | Full Gap     | Mean Cost    | Mean Time    | Note                            |
|------------------------------------------------|--------------|--------------|--------------|---------------------------------|
| $n = 500$ single-head, split-weighted          | 3.193        | –            | –            | earlier split run               |
| $n = 500$ multi-head, split-weighted           | 3.115        | –            | –            | improves over matched $n = 500$ |
| $n = 1000$ single-head, before matched retrain | 3.091        | 76.55        | 3.82s        | earlier single-head model       |
| $n = 1000$ multi-head, unanchored              | 3.134        | 76.65        | 4.02s        | equal-budget but unstable heads |
| $n = 1000$ single-head, matched retrain        | 2.975        | 76.43        | 4.23s        | matched baseline                |
| $n = 1000$ anchored multi-head, 50, 20, 15, 15 | 2.867        | 76.40        | 4.98s        | non-degenerate multi-head       |
| $n = 1000$ anchored multi-head, 97, 1, 1, 1    | <b>2.837</b> | <b>76.35</b> | <b>4.08s</b> | diagnostic                      |

## 5.9 Train-Size Ablation

The evidence distinguishes two failure modes. First, simply adding heads is not enough: the unanchored  $n = 1000$  multi-head model underperforms the earlier single-head model. Second, multi-head guidance becomes useful when one head preserves the strong dynamic prior and the remaining heads add controlled diversity. The 50, 20, 15, 15 allocation is the main completed full-library evidence because it gives half of the ants to non-primary heads while keeping the primary head large enough to preserve the anchored prior. The 97, 1, 1, 1 allocation is reported only to show the upper end of protected-head behavior.

## 6 Mechanism Analysis

The analysis follows the claim spine of the paper rather than adding disconnected diagnostics. The full-library table tests plug-in value: the learned prior improves the same ant-colony process under one-size training. The train-size and anchoring table tests whether the multi-head extension is merely extra capacity; the unanchored loss and anchored win show that the interface must protect a useful dynamic prior before auxiliary heads help. The allocation table tests

the diversity tradeoff directly: equal allocation can be best on small unconstrained instances, while a protected but still non-degenerate allocation gives the strongest completed full-library gap on CVRP. The remaining diagnostics below are therefore not independent claims. They are the measurements needed to explain when the learned priors preserve diversity, when they collapse, and where extra compute is spent.

**Dynamic guidance as a residual intervention.** The central mechanism is that the policy learns a residual intervention on top of distance and pheromone, not a complete solver. This is the reason benchmark transfer is plausible: the search process still enforces feasibility and local projection, while the learned rule changes the local transition distribution using the current solver state. The completed results are consistent with this mechanism because the same trained policy improves over the matched single-head baseline on both full benchmark libraries, despite being trained only on synthetic instances of one size.

**Allocation as robustness–diversity control.** The multi-head results should not be interpreted as “more heads always help.” They show a controlled tradeoff. The 97, 1, 1, 1 diagnostic protects the primary head and gives the best completed CVRP gap, but it is close to single-head behavior. The 50, 20, 15, 15 allocation is the main non-degenerate full-library evidence because half of the ants use auxiliary priors. The 25, 25, 25, 25 allocation shows that the auxiliary heads can carry real search mass, but its advantage is split-dependent rather than universal. The variant evidence identifies the regime in which the mechanism helps and the regime in which diversity becomes dilution.

**Runtime as a mechanism check.** Because multi-head decoding shares one encoder and runs all ant groups in one population update, the expected extra cost should come from head decoding, sparse-prior construction, and dispatching ant groups to their priors. The observed runtime is not uniform across benchmarks: the non-degenerate TSP full-library run is slower than the single-head run, while the completed CVRP allocation comparison shows smaller differences. Runtime decomposition is therefore a mechanism analysis, not an implementation footnote. If most extra time comes from neural generation, the architecture should be simplified. If most extra time comes from sampling or local projection on a few large instances, the tradeoff belongs to the solver interaction.

## 6.1 Head Specialization Analysis

The multi-head component should be analyzed at the level of ant groups, not only by aggregate final gap. For each benchmark bucket, the diagnostic records the mean post-refinement cost produced by each head group across guidance steps. If heads specialize, their relative ranking varies across instance types, search phases, or problem scales. If one head dominates all settings, the multi-head module is interpreted as a protected-primary-prior mechanism rather than as persistent diversity.

Three diagnostics are useful:

1. *Heatmap dissimilarity*: average pairwise cosine distance or rank correlation between head logits on candidate edges.
2. *Selected-edge overlap*: Jaccard overlap among the new edges sampled by ants assigned to different heads.
3. *Head contribution*: fraction of iterations in which each head produces the best ant or improves the incumbent.

These diagnostics connect the method to ACO’s population interpretation. The claim is not merely that there are several neural decoders; the claim is that the colony benefits when different ant groups follow different learned priors inside the same pheromone process.

## 6.2 Stagnation and Anti-Stagnation Analysis

The original conference paper argued that dynamic guidance can counteract pheromone stagnation by suppressing over-reinforced edges. The present paper connects this mechanism to benchmark generalization. For each guidance step, compute the relationship between pheromone rank and neural-logit rank. Two quantities are especially interpretable:

$$\text{Enhance}_q = \frac{|\text{Top}_q(\tau) \cap \text{Top}_q(z)|}{|\text{Top}_q(\tau)|}, \quad (21)$$

and

$$\text{Suppress}_q = \frac{|\text{Top}_q(\tau) \cap \text{Bottom}_q(z)|}{|\text{Top}_q(\tau)|}. \quad (22)$$

Early in search, enhancement may be beneficial because pheromone has not yet collapsed. Later in search, suppression of high-pheromone edges can maintain exploration and prevent the colony from repeatedly sampling the same basin. The analysis should compare this transition on synthetic instances and benchmark-library instances. If the same qualitative transition appears in both settings, it supports the generalization story. If it appears only on synthetic instances, the benchmark-transfer gains should be attributed more narrowly to the heuristic backbone and allocation control.

## 6.3 Runtime Decomposition

The practical concern is whether multi-head guidance preserves an acceptable compute-quality tradeoff. The runtime diagnostic decomposes measured time into:

1. encoder forward pass;
2. head decoder forward pass;
3. sparse-prior construction and transfer;
4. population sampling;
5. scope-restricted local refinement;
6. pheromone update.

Only the decoder, sparse-prior construction, and ant-group dispatch are expected to scale materially with the number of heads. The encoder is shared, and the search process handles all ant groups in one colony batch. If the measured search time grows substantially with the number of heads, the analysis should identify whether the extra cost comes from the learned prior or from downstream search operations on harder instances.

## 6.4 Synthetic Scaling as Secondary Evidence

The conference paper’s synthetic scaling results remain part of the evidence. They answer a different question: whether DYNACO can operate at 1K–100K nodes and whether neural guidance remains computationally lightweight. Here, these results are interpreted after the benchmark-transfer tables or in an extended experiment section. The interpretation also changes. Instead of treating synthetic scaling as proof of generalization, we use it to show that the ACO search process and neural interface remain computationally viable at large  $n$ . Generalization is then established by the benchmark-library results.

## 6.5 Failure-Case Analysis

The benchmark-transfer protocol requires explicit failure analysis. For instances where multi-head guidance loses to single-head guidance or where both lose to unguided ACO, the analysis inspects:

- whether the learned logits over-suppress high-quality pheromone edges;
- whether all heads collapse to similar heatmaps;
- whether demand/capacity structure makes geometric priors misleading;
- whether the candidate graph lacks needed long edges;
- whether runtime is dominated by local repair rather than neural guidance.

This section is especially important because the development path was non-monotone: the first  $n = 1000$  multi-head run lost to single-head guidance, and the later anchored run won on both full benchmark libraries. That nuance strengthens the paper only if it is analyzed directly. The journal claim is not that every multi-head parameterization is better; it is that a population-aligned learned prior can improve transfer when the reliable dynamic prior is preserved and auxiliary priors are given enough, but not excessive, ant mass.

## 7 Discussion

The generalization-centered view changes the interpretation of neural routing progress. Large synthetic instances are still useful, but they are not sufficient evidence that a learned rule is practically general. The benchmark-transfer protocol is stricter because it exposes a trained policy to different scales, geometric structures, and route constraints without retraining. In this setting, dynamic guidance is best understood as a learned intervention inside a general heuristic process. The model is not asked to memorize an entire routing strategy; it learns when and where to bias a strong ACO search process.

Multi-head guidance extends this idea from one learned intervention to a small set of coordinated interventions. This is especially natural for ACO because population diversity is part of the solver’s design. We therefore avoid presenting multi-head guidance as a generic ensemble trick. It is a solver-aligned mechanism: multiple learned heatmaps are mapped to multiple ant groups inside one pheromone process.

## 8 Conclusion

This paper repositions DYNACO around zero-shot benchmark generalization. The method trains on one synthetic size and is evaluated on broad benchmark distributions with gap, runtime, and reliability. The original dynamic guidance mechanism addresses the static-prior mismatch in neural ant-colony search, while the new multi-head component uses the population structure to preserve diverse learned priors in one shared search process. On the completed full-library transfer runs, multi-head guidance with a non-degenerate ant allocation improves over the matched single-head model on both TSPLIB-derived and CVRPLIB-derived benchmarks. The allocation and train-size ablations show that the gain is conditional rather than automatic: heads must preserve a reliable dynamic prior while contributing enough alternative search mass to affect the colony. This turns multi-head guidance from a capacity increase into a test-time population interface for generalization.

## A Benchmark Matrix and Audit Trail

The paper reports completed full-library transfer results for matched  $n = 1000$  single-head and multi-head models on both TSP and CVRP. The saved summaries are the TSP single-head, TSP 50, 20, 15, 15 multi-head, CVRP single-head, and CVRP 50, 20, 15, 15 multi-head JSON files listed in `results.md`. The audit trail also contains the completed allocation-sensitivity summaries used in 2. Unguided ACO rows, bucket-level breakdowns, and per-instance appendix rows should be added under the same inference budget before submission.

## B Default Training and Evaluation Configuration

The default experiment configuration is:

- training size:  $n = 1000$  for the completed full-library runs;
- candidate degree:  $k = 32$ ;
- ants:  $m = 100$ ;
- training horizon:  $H = 10$  outer guidance steps and 100 inner search steps;
- evaluation horizon:  $H = 10$  and 100 inner search steps;
- main multi-head evaluation allocation: 50, 20, 15, 15;
- multi-head deployment: mixed ant groups, all heads active in the same ACO search process.

The  $n = 500$  runs are retained as train-size sensitivity variants rather than as the primary full-library evidence.

## C Detailed Multi-Head Semantics

The multi-head semantics matter because compressing all heads into one heatmap changes the intended algorithm. In the multi-head method, separate head priors are mapped to ant groups. Let  $K_h$  be the number of neural heads and  $m$  be the number of ants. Define a default allocation

$$c_r = \left\lfloor \frac{m}{K_h} \right\rfloor + \mathbf{1}\{r < m \bmod K_h\} \quad (23)$$

as the number of ants assigned to head  $r$  under uniform partitioning. More generally, the allocation vector  $c = (c_1, \dots, c_{K_h})$  may be non-uniform as long as  $c_r \geq 0$  and  $\sum_r c_r = m$ . The search process assigns the first  $c_0$  ants to head 0, the next  $c_1$  ants to head 1, and so on. Ant  $a$  then samples with the heatmap of its assigned head  $g(a)$ :

$$\log p_a(j \mid i) = \alpha \log \tau_{ij} + \beta \log \eta_{ij} + \gamma z_{ij}^{(g(a))}. \quad (24)$$

All ants still contribute to one pheromone field. The head dimension therefore increases colony diversity without running  $K_h$  separate ACO solvers. The same semantics can also be expressed with one prior per ant; the reported multi-head setting uses one transition distribution per head.

For capacitated routing, the same semantics apply, except the feasible set is additionally masked by route capacity and depot-transition rules. The neural head does not decide feasibility. It only scores candidate edges that the search process marks feasible. This preserves the solver’s hard-constraint handling and keeps the multi-head module problem-agnostic at the policy level.

## D Extended Table Set

The extended result set is organized around the following evidence:

1. **Main benchmark transfer:** routing buckets, unguided ACO, single-head guidance, and multi-head guidance, with gap/time/reliability.
2. **External survey comparison:** selected neural routing solvers and classical heuristics from the survey pipeline, separated from internal ablations if inference budgets differ.
3. **Train-size ablation:**  $n = 500$  versus  $n = 1000$  for single-head and multi-head models.
4. **Head-count ablation:** one head, two heads, four heads, and eight heads under the same total ant count.
5. **Runtime decomposition:** encoder, decoder, sparse-prior construction, population sampling, local refinement, pheromone update.
6. **Synthetic scaling:** retained from the conference paper but moved after benchmark generalization.
7. **Per-instance appendix:** full benchmark-library rows.

## References

- [1] M. Dorigo, V. Maniezzo, and A. Coloni. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics*, 1996.
- [2] T. Stutzle and H. H. Hoos. MAX-MIN ant system. *Future Generation Computer Systems*, 2000.
- [3] G. Reinelt. TSPLIB: a traveling salesman problem library. *ORSA Journal on Computing*, 1991.
- [4] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 2017.
- [5] K. Helsgaun. An effective implementation of the Lin-Kernighan traveling salesman heuristic. *European Journal of Operational Research*, 2000.
- [6] T. Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP\* neighborhood. *Computers & Operations Research*, 2022.
- [7] Y. Ye, J. Wang, Z. Cao, H. Liang, and Y. Li. DeepACO: Neural-enhanced ant systems for combinatorial optimization. In *NeurIPS*, 2023.
- [8] W. Zhang, Z. Cong, C. Hua, and X. Liu. GFACS: GFlowNet-based ant colony sampling for combinatorial optimization. In *NeurIPS*, 2023.
- [9] Y. Li, J. Guo, R. Wang, and J. Yan. From distribution learning in training to gradient search in testing for combinatorial optimization. In *NeurIPS*, 2023.
- [10] Y. Chen, X. Zhang, and L. Song. Heatmap-guided ant colony optimization for neural combinatorial optimization. In *ICLR*, 2024.
- [11] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *NeurIPS*, 2020.

- [12] Q. Drakulic, S. Michel, F. Mai, A. Sors, and J.-M. Andreoli. BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization. In *NeurIPS*, 2023.
- [13] Y. Luo, Z. Chen, J. Li, and W. Zhang. Neural combinatorial optimization with heavy decoder: toward large scale generalization. In *NeurIPS*, 2023.
- [14] S. Li, Z. Yan, and C. Wu. Learning to delegate for large-scale vehicle routing. In *NeurIPS*, 2021.
- [15] CIAM Group. Neural routing solvers: a survey with a generalization-focused evaluation pipeline. Technical report, 2026.